CI/CD Pipeline Documentation

Automated Build and Deployment to Self-Hosted Linux Server

Document Version: 1.0
Last Updated: March 2025
Application: Daily Post App (.NET Core)

Developed by MD Yunus Ali Rony
Software Development Engineer ||
Email: talukder.roni.ict@gmail.com

Table of Contents

Contents

Overview

Purpose

This document provides a complete guide to setting up an automated **CI/CD pipeline** using **GitHub Actions**, **Docker**, and a **self-hosted Linux server**.
The pipeline builds, containerizes, and deploys the application automatically on every push to the main branch.

Workflow Summary

Developer Push
→ GitHub Actions Trigger
→ Build Docker Image
→ Push Image to GitHub Container Registry (GHCR)
→ Self-Hosted Runner Pulls Image
→ Deploys Container on Linux Server

STEP 1: Prerequisites

Development Machine

- Docker Desktop installed and running
- Access to GitHub repository

Linux Server Requirements

- Ubuntu 20.04 LTS or later
- Docker Engine installed
- Minimum **2 GB RAM** and **20 GB disk space**
- Outbound internet access (HTTPS – port 443)
- User with sudo privileges

GitHub Requirements

- GitHub repository with write access

- Access to **Settings → Actions → Runners**
- GitHub Packages (GHCR) enabled

STEP 2: Dockerization on Local Machine

The first step is to containerize the application using Docker.

Dockerfile Overview

The Dockerfile defines how the application is built and run inside a container.
A **multi-stage build** is used to reduce final image size and improve security.

Dockerfile Example (.NET Application)

Create a Dockerfile inside the Source-Code directory.

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY *.csproj ./
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
RUN adduser --disabled-password --gecos '' appuser
COPY --from=build /app/publish .
RUN chown -R appuser:appuser /app
USER appuser
EXPOSE 8080
ENTRYPOINT ["dotnet", "YourApplication.dll"]
```

Note: Dockerfile content may vary slightly based on application type.

STEP 3: Local Testing with Docker Desktop

Before CI/CD, always validate the Docker image locally.

Build Image Locally

```
docker build -t daily-post-app:local .
docker images
```

Run Container Locally

```
docker run -d \
  --name daily-post-local \
  -p 8000:8080 \
  -e ASPNETCORE_ENVIRONMENT=Development \
  daily-post-app:local
```

Verify Application

- Browser: http://localhost:8000
- Logs:

```
docker logs daily-post-local
```

Cleanup

```
docker stop daily-post-local
docker rm daily-post-local
docker rmi daily-post-app:local
```

STEP 4: Server Readiness & Docker Installation

Connect to Server

```
ssh username@server-ip
```

Verify Docker Installation

```
docker --version
```

If Docker is not installed, install it:

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
sudo systemctl enable docker
sudo systemctl start docker
```

Add User to Docker Group

```
sudo usermod -aG docker username
```

Re-login to apply changes.

STEP 5: GitHub Self-Hosted Runner Setup

A **self-hosted runner** is critical because deployment happens directly on the server.

Create Runner in GitHub

Navigate to:
Repository → Settings → Actions → Runners → New self-hosted runner

Select:

- OS: Linux
- Architecture: x64

Copy the configuration commands shown by GitHub.

Configure Runner on Server

```
mkdir actions-runner
cd actions-runner
```

Run all commands copied from GitHub.

Recommended inputs:

- Runner name: uat-docker-runner-01
- Labels: docker,linux,uat
- Work folder: default (_work)

Install Runner as a Service

```
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

Runner should show **Active (running)**.

STEP 6: SSH Key Generation and Secure Storage in GitHub

Secure server access is mandatory for safe and automated deployments.
This pipeline uses **SSH key-based authentication** instead of passwords.

SSH Key Generation on the Server

SSH keys must always be generated **on the Linux server**, not on a local machine.

Run the following command on the server:

```
ssh-keygen -t ed25519 -C "github-ci-cd-deploy"
```

When prompted:

- File location: press **Enter** to use default path (~/.ssh/id_ed25519)
- Passphrase: optional (recommended for production)

This creates:

- **Private Key**: ~/.ssh/id_ed25519
- **Public Key**: ~/.ssh/id_ed25519.pub

Configure Authorized Keys

Add the public key to the authorized keys file:

```
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
```

Set correct permissions:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

This allows secure, passwordless SSH access to the server.

Verify SSH Access

From the server itself or another machine:

```
ssh username@server-ip
```

If login works without asking for a password, SSH configuration is successful.

Store SSH Credentials in GitHub Secrets

The **private SSH key must never be committed** to the repository.

Copy the private key content:

```
cat ~/.ssh/id_ed25519
```

Go to:

GitHub Repository → Settings → Secrets and variables → Actions

Add the following **GitHub Secrets**:

- SSH_PRIVATE_KEY
  → Paste full private key content
- SSH_USERNAME
  → Linux server username (example: admin)
- SERVER_HOST
  → Server IP or DNS name (example: 192.168.1.100)

STEP 7: GitHub Actions CI/CD Pipeline

Complete Pipeline:

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  build:
    name: Build and Push Docker Image
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    outputs:
      image-tag: ${{ steps.meta.outputs.tags }}
      image-digest: ${{ steps.build.outputs.digest }}

    steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Log in to Container Registry
      uses: docker/login-action@v3
      with:
        registry: ${{ env.REGISTRY }}
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    - name: Extract metadata
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
```

```
      tags: |
        type=ref,event=branch
        type=ref,event=pr
        type=sha,prefix={{branch}}-
        latest

  - name: Build and push Docker image
    id: build
    uses: docker/build-push-action@v5
    with:
      context: ./Source-Code
      file: ./Source-Code/Dockerfile
      push: true
      tags: ${{ steps.meta.outputs.tags }}
      labels: ${{ steps.meta.outputs.labels }}

deploy:
  name: Deploy to Server
  needs: build
  runs-on: self-hosted   # This will use your uat-fe4-docker-02 runner
  if: github.ref == 'refs/heads/main'

  steps:
  - name: Deploy application via SSH
run: |
  # Create temporary key file
  echo "${{ secrets.SSH_PRIVATE_KEY }}" > /tmp/id_rsa.pem
  chmod 600 /tmp/id_rsa.pem

  # SSH and run deployment commands
  ssh -i /tmp/id_rsa.pem -o StrictHostKeyChecking=no \
      ${{ secrets.SSH_USERNAME }}@${{ secrets.SERVER_HOST }} << 'EOF'

    docker login ghcr.io -u ${{ github.actor }} -p ${{ secrets.GITHUB_TOKEN }}

    docker stop daily-post-app || true
    docker rm daily-post-app || true

    docker pull ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:latest
    docker run -d \
      --name daily-post-app \
      --restart unless-stopped \
      -e ASPNETCORE_ENVIRONMENT=uat \
      -v ${{ vars.LOG_PATH }}:/app/Logs \
      -p 8000:8080 \
```

```
        ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:latest


      docker image prune -f
    EOF

    # Remove temporary key file
    rm -f /tmp/id_rsa.pem
```

Pipeline Overview

The pipeline has **two jobs**:

Build Job

- Runs on GitHub-hosted runner
- Builds Docker image
- Pushes image to GHCR

Deploy Job

- Runs on self-hosted runner
- Pulls latest image
- Deploys container on Linux server

Environment & Secrets

The following values should be stored in **GitHub Secrets / Variables**:

- LOG_PATH → /var/daily-post/logs
- SSH_KEY_PATH → Used for secure server access (if required)

These values can be reused across environments without hardcoding paths.

STEP 7: Deployment Verification

GitHub Actions Verification

- Go to **Repository → Actions**
- Confirm workflow completed successfully

Server Verification

```
docker ps
docker logs daily-post-app
```

Application Verification

```
curl -I http://localhost:8000
```

Expected result:

```
HTTP/1.1 200 OK
```

Important Considerations & Best Practices

- Always test Dockerfile locally before CI/CD
- Store sensitive values in **GitHub Secrets**
- Use volume mounts for logs
- Monitor disk usage on server
- Avoid relying only on `latest` tag in production
- Regularly prune unused Docker images
- Keep runner always running as a system service